

Sorting and searching

LEARNING OBJECTIVES

After studying this chapter, the readers will be able to:

- Understand different sorting algorithms
- Implement different sorting algorithms
- Understand different searching algorithms
- Implement different searching algorithms

Sorting is the process of placing an existing group of components in a specific order. A group of integers can be arranged in either ascending (growing) or descending (decreasing) order. If the collection consists of strings, we can arrange them alphabetically (from a to z or z to a) or by string length. For instance, a dictionary will list its words in alphabetical order.

Finding a specific element within a group of components is the definition of searching. Whether an element is present in the collection or not is determined by the search result. If it is, we may also determine where that element is in the specified collection. A key method in computer science is searching. Programmers must be aware of the many methods for searching and retrieving data in order to create algorithms.

In this chapter, we will learn about a few sorting and searching methods and implement them using Python.

9.1 INTRODUCTION TO SORTING

Sorting is a fundamental computer science function that entails putting a group of objects or data pieces in a particular order. It is essential to many applications and algorithms because it makes data retrieval, organizing, and analysis more effective. Sorting algorithms are crucial tools in computer programming, whether they are used to arrange a list of names in alphabetical order or to sort a large data set for analysis.

The primary objective of sorting is to rearrange the items according to a predetermined comparison criterion, usually in ascending or decreasing order. This order may be determined by a number of factors, including timestamps, numerical values, alphabetical order or user-defined criteria.

There are numerous types of sorting algorithms, each with distinct advantages and disadvantages. While some algorithms promote effectiveness and ideal performance, others place a higher priority on simplicity and ease of implementation. The amount of the data set, the preferred order and the available processing resources are only a few examples of the variables that influence the choice of sorting algorithm.

Commonly used sorting algorithms include:

1. Bubble sort: A straightforward method that checks nearby elements frequently and swaps them if the order is incorrect.
2. Insertion sort: Through iteratively inserting components into the appropriate locations, this technique creates a sorted subsequence.
3. Selection sort: An algorithm that finds the smallest (or largest) element in an unsorted list and places it in the correct position.
4. Merge sort: This algorithm employs the divide-and-conquer strategy, separating the list recursively into smaller sublists, sorting them and then merging them back together.
5. Quick sort: Choosing a pivot element, dividing the list around it and then recursively sorting the resulting sublists are all aspects of another divide-and-conquer method.
6. Heap sort: This algorithm repeatedly extracts the maximum (or minimum) element from a binary heap data structure to sort the elements.

In this chapter, we will discuss the first three sorting algorithms.

A key component of organizing and arranging data in a certain order is sorting, a fundamental idea in computer science. In numerous applications and algorithms, it is a crucial process that enables effective searching, data analysis and problem-solving. Sorting includes placing a group of items in a particular order, usually according to their values or other predetermined criteria.

Sorting is important when dealing with large amounts of data that need to be sorted, such as a list of names, a database of client data or a collection of numbers. Sorting the data allows us to expedite processes, enhance search results and gain valuable insights from the ordered data.

There are numerous sorting algorithms, and each has advantages and disadvantages in terms of productivity, stability and application for particular data sets. Some of the most well-known sorting algorithms include bubble sort, selection sort, insertion sort, merge sort, quick sort and heap sort. Each algorithm follows a specific set of steps to rearrange the pieces and generate the necessary sorted output.

The size of the data collection, how the data are distributed, memory requirements and desired performance characteristics all play a role in selecting the best sorting method. To choose an algorithm that satisfies the particular needs of the current problem, it is critical to take these elements into account.

9.1.1 Bubble sort

The first sorting method we'll learn about is the bubble sort. By continually comparing neighboring elements and swapping them if they are unordered, it sorts a given list of elements. When two items are swapped, their relative positions are also changed. *Pass* is the term used in algorithms to describe each iteration of a list's elements. The bubble sort sorts a list with n elements over a total of $n - 1$ passes. The necessary adjacent pairs of list entries will be compared in each pass. The largest element is determined after each pass and positioned at the appropriate location in the list in order to arrange the components in ascending order. This can be considered as the largest element being "bubbled up." Hence the name bubble sort. This sorted element is not considered in the remaining passes and thus the list of elements gets reduced in successive passes.

Algorithm 1

```

1. BUBBLESORT( numList, n)
2. SET i = 0
3. WHILE i < n REPEAT STEPS 3 to 8
4.   SET j = 0
5.   WHILE j < n-i-1, REPEAT STEPS 5 to 7
6.     IF numList[j] > numList[j+1] THEN
7.       swap(numList[j], numList[j+1])
8.     SET j=j+1
9. SET i=i+1

```

Example

Sort the elements in Figure 9.1a using the bubble sort algorithm.

List	13	12	18	6	-4	9
Index	0	1	2	3	4	5

Figure 9.1a Bubble sort

Comparison in Pass 1 (Figure 9.1b).

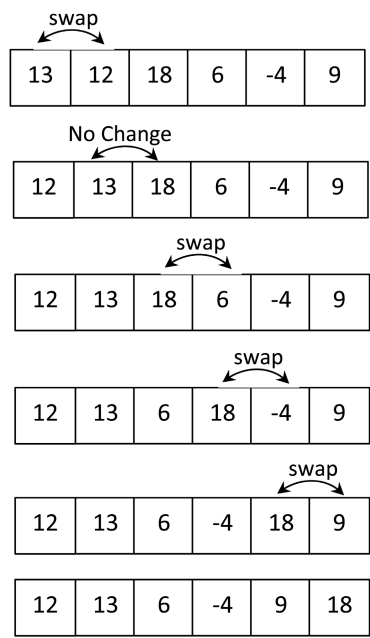


Figure 9.1b Bubble sort – Comparison in Pass 1

Comparison in Pass 2 (Figure 9.1c).

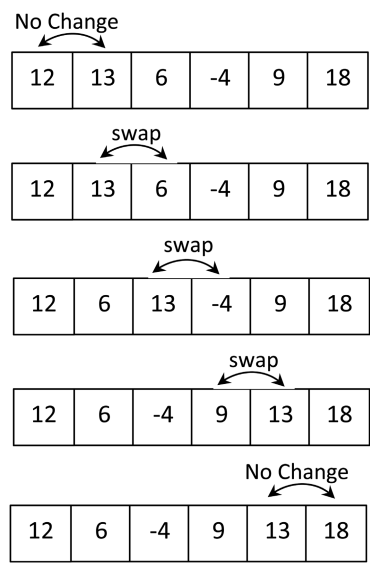


Figure 9.1c Bubble sort – Comparison in Pass 2

Comparison in Pass 3 (Figure 9.1d).

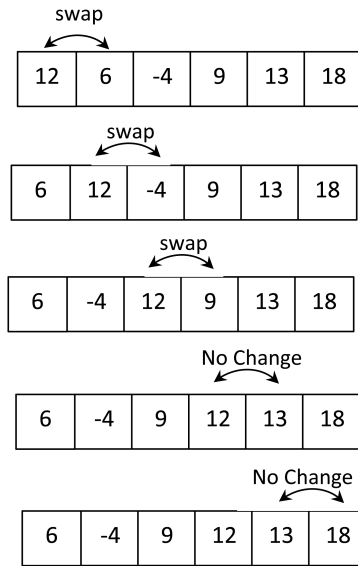


Figure 9.1d Bubble sort – Comparison in Pass 3

Comparison in Pass 4 (Figure 9.1e).

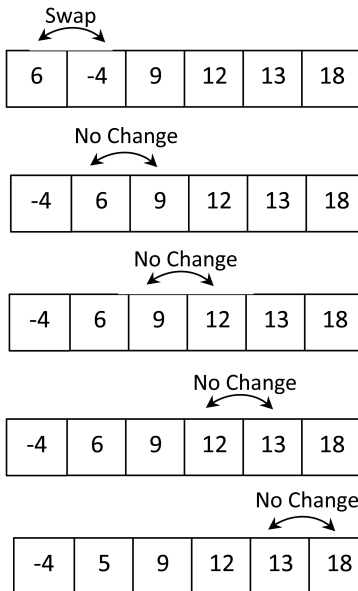


Figure 9.1e Bubble sort – Comparison in Pass 4

Comparison in Pass 5 (Figure 9.1f).

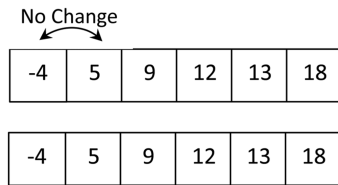


Figure 9.1f Bubble sort – Comparison in Pass 5

Python program 1

```
def bubblesort(elements):
    swapped = False
    # Looping from size of array from last index[-1] to index [0]
    for n in range(len(elements)-1, 0, -1):
        for i in range(n):
            if elements[i] > elements[i + 1]:
                swapped = True
                # swapping data if the element is less than next
                # element in the array
                elements[i], elements[i + 1] = elements[i + 1],
                elements[i]
            if not swapped:
                # exiting the function if we didn't make a single swap
                # meaning that the array is already sorted.
                return

elements = [13, 12, 18, 6, -4, 9]

print("Unsorted list is,")
print(elements)
bubblesort(elements)
print("Sorted Array is, ")
print(elements)
```

Output

```
Unsorted list is,
[13, 12, 18, 6, -4, 9]
Sorted Array is,
[-4, 6, 9, 12, 13, 18]
```

9.1.2 Insertion sort

Another sorting technique that can place items in a given list in either ascending or descending order is insertion sort. Similar to selection sort,

insertion sort divides the list into two sections: one with sorted entries and the other with unsorted elements. One by one, each item in the unsorted list is taken into account and added to the sorted list in the proper location. The sorted list is walked through once for each pass to locate the potential spot to insert an unsorted element. Thus, the sorting technique is known as insertion sort.

In pass 2, starting with element *e* of the unsorted list in the backward direction, each element of the sorted list will be compared with element *e* of the unsorted list until the proper position for insertion is identified. The components of the sorted list will be moved to the right, creating room for element *e* to be entered.

This keeps happening until every element from unsorted lists has been added to the sorted list at the proper places.

As a consequence, a sorted list is produced, with the components organized in ascending order.

Algorithm 2

```

INSERTIONSORT( numList, n)
1. i=1
2. WHILE i< n REPEAT STEPS 3 to 9
3. temp = numList[i]
4. j = i-1
5. WHILE j>= 0 and numList[j]>temp, REPEAT STEPS 6 to 7
    numList[j+1] = numList[j]
    j=j-1
6. numList[j+1] = temp #insert temp at position j
7. i=i+1

```

Example

Sort the data in Figure 9.2a using insertion sort.

List	13	12	18	6	-4	9
Index	0	1	2	3	4	5

Figure 9.2a Insertion sort

Comparison in Pass 1: At the beginning, 13 was in the list; next 12 is inserted into it (Figure 9.2b).

Comparison in Pass 2: The next data 18 is inserted into the list (Figure 9.2c).

Comparison in Pass 3: The next data 6 is inserted into the list (Figure 9.2d).

Comparison in Pass 4: The next data -4 is inserted into the list (Figure 9.2e).

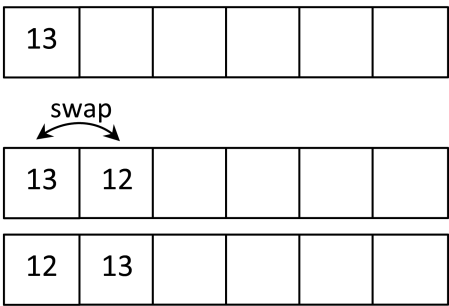


Figure 9.2b Insertion sort – Comparison in Pass 1

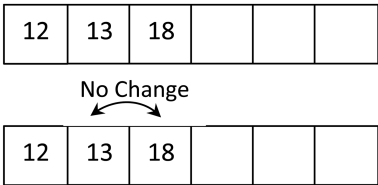


Figure 9.2c Insertion sort – Comparison in Pass 2

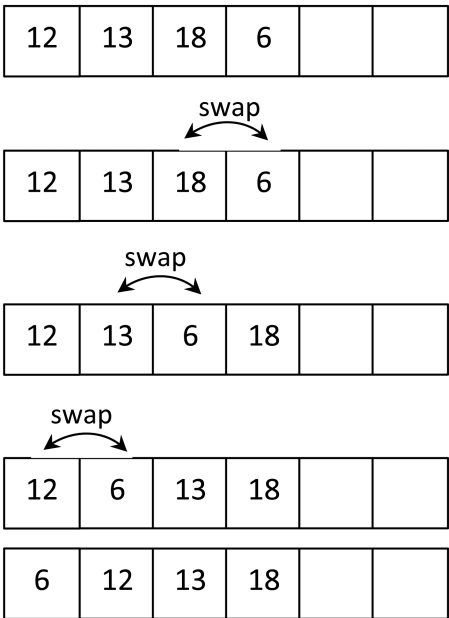


Figure 9.2d Insertion sort – Comparison in Pass 3

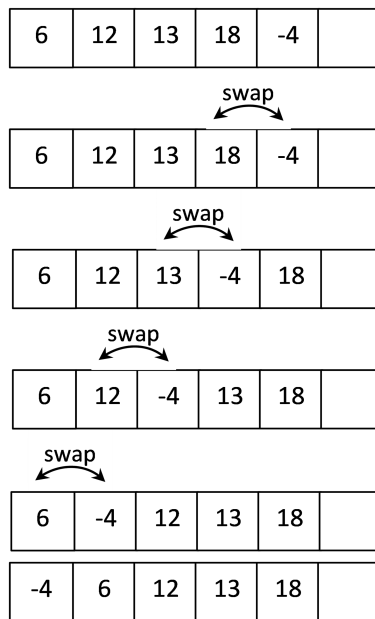


Figure 9.2e Insertion sort – Comparison in Pass 4

Comparison in Pass 5: The next data 9 is inserted into the list (Figure 9.2f).

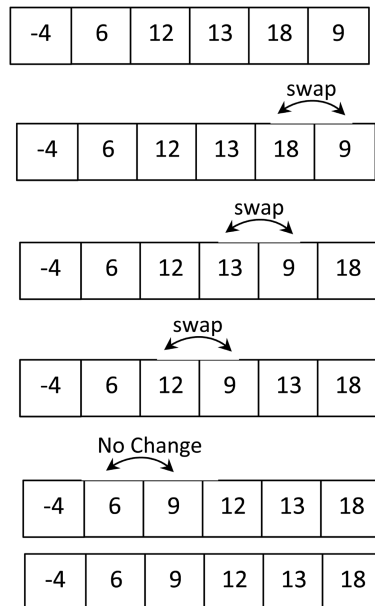


Figure 9.2f Insertion sort – Comparison in Pass 5

Python program 2

```
# Function to do insertion sort
def insertionSort(arr):

    # Traverse through 1 to len(arr)
    for i in range(1, len(arr)):

        key = arr[i]

        # Move elements of arr[0..i-1], that are
        # greater than key, to one position ahead
        # of their current position
        j = i-1
        while j >=0 and key < arr[j] :
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key

# Driver code to test above
arr = [13, 12, 18, 6, -4, 9]
print("Unsorted list is,")
print (arr)
insertionSort(arr)
print ("Sorted array is:")
print (arr)
```

Output

```
Unsorted list is,
[13, 12, 18, 6, -4, 9]
Sorted array is:
[-4, 6, 9, 12, 13, 18]
```

9.1.3 Selection sort

Another sorting method is selection sort. The selection sort makes $(n - 1)$ number of trips through the list in order to sort a list with n elements. The list is thought to be split into two lists, with the right list containing the unsorted elements and the left list carrying the sorted elements. The right list initially has all the elements, but the left list is initially empty.

In the first pass, all of the elements in the unsorted list are scanned to identify the smallest element in order to arrange the elements in ascending order. The leftmost element of the unsorted list is then switched for the smallest element. This element is at the top spot in the sorted list and is skipped over in subsequent passes. The leftmost member of the unsorted list is swapped with the next-smallest element in the second pass, which is chosen from the remaining items in the unsorted list.

This element is the second item in the sorted list, and during the third run, the unsorted list is reduced by one element. This procedure is repeated until the $n-1$ smallest elements are located and placed in the appropriate locations. The final and previously present ingredient is the n th one.

Algorithm 3

```

SELECTIONSORT( numList, n)
1. i=0
2. WHILE i< n REPEAT STEPS 3 to 11
3.   min = i, flag = 0
4.   j= i+1
5.   WHILE j< num REPEAT STEPS 6 to 10
6.     IF numList[j] < numList[min] THEN
7.       min = j
8.       flag = 1
9.   IF flag = 1 THEN
10.    swap(numList[i], numList[min])
11. i=i+1

```

Example

Sort the data in Figure 9.3a using insertion sort.

List	13	12	18	6	-4	9
Index	0	1	2	3	4	5

Figure 9.3a Selection sort

Comparison in Pass 1 (Figure 9.3b).

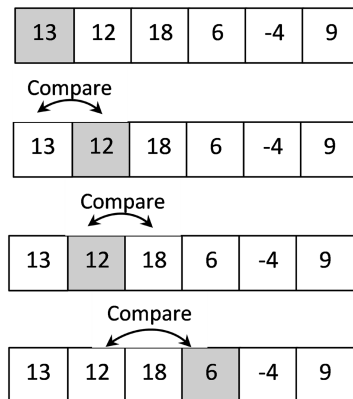


Figure 9.3b Selection sort – Comparison in Pass 1

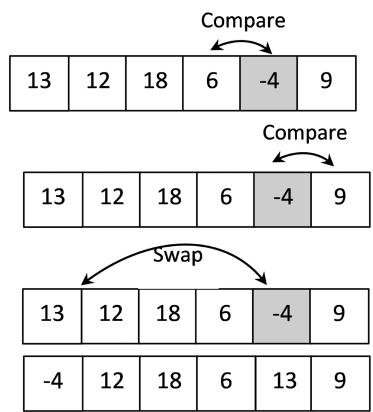


Figure 9.3b Continued

Comparison in Pass 2 (Figure 9.3c).

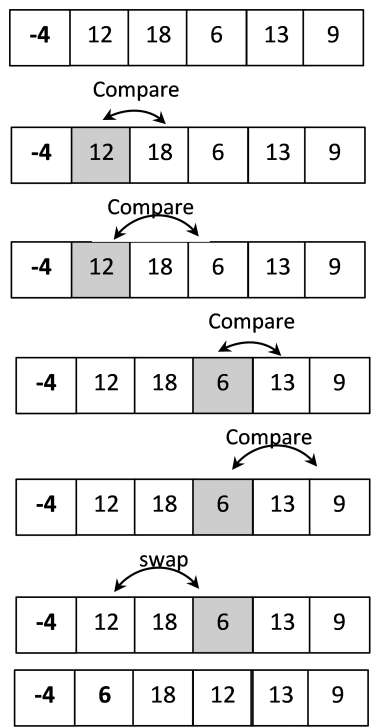


Figure 9.3c Selection sort – Comparison in Pass 2

Comparison in Pass 3 (Figure 9.3d).

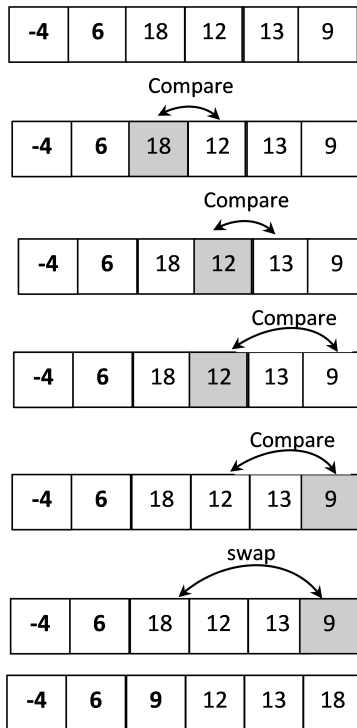


Figure 9.3d Selection sort – Comparison in Pass 3

Comparison in Pass 4 (Figure 9.3e).

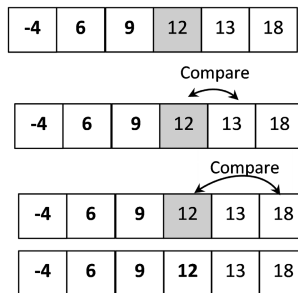
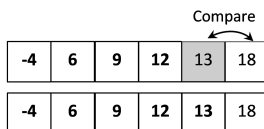


Figure 9.3e Selection sort – Comparison in Pass 4

Comparison in Pass 5 (Figure 9.3f).



-4	6	9	12	13	18
-4	6	9	12	13	18

Figure 9.3f Selection sort – Comparison in Pass 5

Python program 3

```
A = [13, 12, 18, 6, -4, 9]
print ("UnSorted array")
print(A)
# Traverse through all array elements
for i in range(len(A)):

    # Find the minimum element in remaining
    # unsorted array
    min_idx = i
    for j in range(i+1, len(A)):
        if A[min_idx] > A[j]:
            min_idx = j

    # Swap the found minimum element with
    # the first element
    A[i], A[min_idx] = A[min_idx], A[i]

# Driver code to test above

print ("Sorted array")
print(A)
```

Output

```
UnSorted array
[13, 12, 18, 6, -4, 9]
Sorted array
[-4, 6, 9, 12, 13, 18]
```

9.2 INTRODUCTION TO SEARCHING

The practice of choosing specific information based on predetermined criteria from a set of data is called searching. This idea is known to you from your experiences searching the web for sites that include specific words or phrases or from looking up phone numbers in phone books. We limit the

definition of searching in this article to the act of locating a particular item within a collection of data items.

There are several data structures on which the search function can be used. The main topic of this chapter is sequence search, which is locating a specific item inside a sequence by employing a search key. The data items in a collection are identified by a key, which is a distinct value. The values in a collection of basic types, such as integers or reals, are the keys.

It is necessary to determine which particular data component is the key for collections of complicated kinds. Compound keys, as they are sometimes called, are made up of several different parts.

9.2.1 Linear search

Among search techniques, linear search is the most basic and straightforward. It is a thorough search method in which each entry in a list is compared to the object you're looking for, also known as the "key." As a result, the key is compared one by one with each member in the list. This procedure is repeated until an element that matches the key is located, at which point we proclaim the search to be successful. We deem the search failed if, after searching the whole list, no element matches the key, indicating that the key is not present in the list. The order in which the items are contained in the list is followed when comparing each item individually, starting with the first and working all the way down to the last. As a result, it is also known as a serial or sequential search. When gathering little, unordered things, this strategy might be helpful.

Algorithm 4

```
LinearSearch(numList, key, n)
1. index = 0
2. WHILE index < n, REPEAT Step 3
3.     IF numlist[index] = key THEN
4.         PRINT "Element found at position", index+1 STOP
5.     ELSE index = index+1
6. PRINT "Search unsuccessful"
```

Example

Search for key = 6 in the given list.

List	13	12	18	6	-4	9
Index	0	1	2	3	4	5

Here $n = 6$ (number of data available) (Table 9.1).

Table 9.1 Linear search

Index	Index < n	numList[index] = key	Index = index + 1
0	0 < 6 ? Yes	13 = 6 ? No	1
1	1 < 6 ? yes	12 = 6 ? No	2
2	2 < 6 ? Yes	18 = 6 ? No	3
3	3 < 6 ? Yes	6 = 6 ? yes	

Observe that after four comparisons, the algorithm found the key 17 and will display “Element found at position 4”.

Let us search for key = 17 (Table 9.2).

Table 9.2 Linear search

Index	Index < n	numList[index] = key	Index = index + 1
0	0 < 6 ? Yes	13 = 17 ? No	1
1	1 < 6 ? Yes	12 = 17 ? No	2
2	2 < 6 ? Yes	18 = 17 ? No	3
3	3 < 6 ? Yes	6 = 17 ? No	4
4	4 < 6 ? Yes	-4 = 17 ? No	5
5	5 < 6 ? Yes	9 = 17 ? No	6
6	6 < 6 ? No		

Observe that after six comparisons, the algorithm does not find the key 17 and will display “Search unsuccessful”.

Python program 4

```
def linear_Search(list1, n, key):  
    # Searching list1 sequentially  
    for i in range(0, n):  
        if (list1[i] == key):  
            return i  
    return -1  
  
list1 = [13, 12, 18, 6, -4, 9]  
key = 6  
  
n = len(list1)  
res = linear_Search(list1, n, key)  
if(res == -1):  
    print("Search unsuccessful")  
else:  
    print("Element",key,"found at index: ", res)
```

Output

Element 6 found at index: 3

9.2.2 Binary search

The binary search is a method of rapidly finding a key by using the ordering of the list's members. The list's members can be ordered according to their key values in either ascending or descending order for numeric values. Textual data can either be sorted alphabetically from z to a or from a to z.

In a binary search, the element in the center of a sorted list is compared with the key to be searched. Any one of the following three outcomes might arise from this:

- i) The element in the middle place matches the key.
- ii) The element is larger than the key.
- iii) The element is smaller than the key.

The search is deemed successful and comes to an end if the element in the middle position matches the key.

If the middle element exceeds the key, the key must unquestionably be in the first half of the list, if it is present at all. Therefore, we may skip over the second half of the list right away and limit our search to the first half.

If the middle element is smaller than the key, the key must be in the second half of the list, if it is there at all. Thus, we may skip over the first half of the list right away and limit our search to the second half. Until the key was located or the remaining list had just one item, this splitting and list-size reduction process is carried out. The search fails if that item is not the key since the key is not in the list.

Therefore, it is clear that a binary search can be more effective than a linear search of individual components if the list we need to search is organized in a certain way. In other words, the list must be sorted. The floor division ($//$) operator is used to determine the mid value if the list to be searched has an even number of entries. The midpoint (mid) of the list, if there are 10 entries, is equal to $10//2$, or 5. Since the first element in the list has an index value of 0, the sixth element in the list is thus regarded as the middle element. If required, the list is further divided into two parts where the first half contains five elements and the second half contains four elements.

Interestingly, the intermediate comparisons that do not locate the key nonetheless provide information about the section of the list that could contain the key! They indicate whether the key is located before or after the list's current middle position, and we make use of this information to limit the scope of our search. The term *binary search* comes from the fact that every failed comparison cuts the amount of components that need to be searched by half. Now let's talk about the binary search algorithm.

Algorithm 5

```
BinarySearch(numList, key)
1. first = 0, last = n-1
2. mid = (first+last)//2
3. WHILE first <= last REPEAT Step 4
4. IF numList[mid] = key THEN
    PRINT "Element found at position", " mid+1
    STOP
    ELSE IF numList[mid] > key, THEN
        last = mid-1 ELSE first = mid + 1
5. PRINT "Search unsuccessful"
```

Example

Consider a sorted list consisting of 15 elements: numList = [2, 3, 5, 7, 10, 11, 12]. The key = 10, has to be found in numList. The index values of the first, middle and final elements are found in the numList.

Search for key = 10 in the given list. (Also see Tables 9.3 and 9.4.)

List	2	3	5	7	10	11	12
Index	0	1	2	3	4	5	6

Table 9.3 Binary search – data

	First			Mid			Last
Index	0	1	2	3	4	5	6
numList[]	1	2	5	7	10	11	12

Table 9.4 Binary search – search key =10

	First	Last	Mid	numList [mid]==key ?	Key < numList[mid] ?	First <= last
Iteration 0	0	6	(0+6)//2=3	7 == 10 No	10 < 7 No	0 <= 6 Yes
Iteration 1	4	6	(4+6)//2=5	11 == 10 No	10 < 11 Yes	4 <= 6 Yes
Iteration 2	4	5	(4+5)//2 = 4	10 == 10 Yes		

Python program 5

```
def binary_search(arr, low, high, x):
    # Check base case
    if high >= low:
        mid = (high + low) // 2
        # If element is present at the middle itself
        if arr[mid] == x:
            return mid
```

```

    # If element is smaller than mid, then it can only
    # be present in left subarray
    elif arr[mid] > x:
        return binary_search(arr, low, mid - 1, x)

    # Else the element can only be present in right subarray
    else:
        return binary_search(arr, mid + 1, high, x)

else:
    # Element is not present in the array
    return -1

# Test array
arr = [2, 3, 5, 7, 10, 11, 12]
x = 10

# Function call
result = binary_search(arr, 0, len(arr)-1, x)

if result != -1:
    print("Element",x,"is present at index", str(result))
else:
    print("Element is not present in array")

```

Output

Element 10 is present at index 4

True–false questions

- The `sort()` method is valuable because it keeps elements of parallel arrays in the correct order.
 - True
 - False
- The given array is `arr = {1,2,4,3}`. Bubble sort is used to sort the array elements. Two iterations are required to sort the array with an improvised version.
 - True
 - False
- The advantage of selection sort over other sorting techniques is that it does not require additional storage space.
 - True
 - False
- When the list has only a few elements and when performing a single search in an unordered list, then a linear search is used.
 - True
 - False
- Given `{5,6,7,8,9}` and `key = 8`, three iterations are done until the element is found using binary search.
 - True
 - False

Fill-in-the-blank questions

1. For the given array {45,77,89,90,94,99,100} with searching key = 100, _____ and _____ are the mid values generated in the first and second iterations.
2. The number of iterations in bubble sort and selection sort, respectively, for the array {3,4,5,2,1} are _____ and _____.
3. For the array {1, 2, 4, 3}, to sort the items using bubble sort _____ iterations are required.
4. The linear search (recursive) algorithm is used _____.
5. For the array {1,2,3,6,8,10} with search key 17, _____ calls are required to confirm an unsuccessful search.

Multiple-choice questions

1. For the array {25, 45, 87, 21, 18, 49, 13, 115, 83, 65}, how many searches are required to find 83?
 - a. 8
 - b. 9
 - c. 10
 - d. 11
2. What benefit does a recursive approach have over an iterative one?
 - a. Requires more memory
 - b. Uses less memory
 - c. Requires more code to be written
 - d. Requires less code and is simple to implement
3. Out of the following, which one does not involve the use of binary search?
 - a. Union of intervals
 - b. To look through an unsorted list
 - c. Debugging
 - d. Determining the top and lower bounds of an ordered sequence
4. Linear search is also called
 - a. Perfect search
 - b. Random search
 - c. Sequential search
 - d. None of the above
5. The necessary condition for using binary search in an array is
 - a. The array should of more size
 - b. The array should not be too long
 - c. The array should be sorted
 - d. None of the above

- ## Descriptive questions

1. Consider a list of ten elements: numList = [7, 11, 3, 10, 17, 23, 1, 4, 21, 5]. Display the partially sorted list after three complete passes of bubble sort.
2. Determine the number of swaps needed to sort the following list using bubble and selection sort, and determine which sorting method is more effective in terms of comparisons made (2, 5, 34, 8, 90, -33, 43, -22, 32).

3. Students' names are entered into a course in increasing order at the time of acceptance. The sorting process is completed concurrently with adding entries to a list. Determine the kind of sorting method being employed and create a program with a user-defined function that saves names in the list in ascending order each time a name is entered.
4. Using a linear search, determine the position of 8, 1, 99 and 44 in the list: [1, -2, 32, 8, 17, 19, 42, 13, 0, 44]. Draw a detailed table showing the values of the variables and the decisions taken in each pass of the linear search.
5. Write a program that takes as input a list having a mix of ten negative and positive numbers and a key value. Apply a linear search to find whether the key is present in the list. If the key is present, it should display the position of the key in the list; otherwise it should print an appropriate message. Run the program for at least three different keys and note the result.

Answers to true-false questions

1. False
2. True
3. True
4. True
5. False

Answers to fill-in-the-blank questions

1. 90, 99
2. 5, 4
3. four
4. when the size of the dataset is low
5. seven

Answers to multiple-choice questions

- | | | | | | | | | | |
|----|---|----|---|----|---|----|---|-----|---|
| 1. | b | 2. | d | 3. | b | 4. | c | 5. | c |
| 6. | a | 7. | b | 8. | a | 9. | d | 10. | b |